

SOLID Principles & SRP

SWEN1 | BIF3

Table of Contents

- SOLID Principles
- Single-Responsibility-Principle



SOLID Principles

Why SOLID still matters

Motivation: The Cost of Change

- 70 % of lifetime cost = maintenance & evolution
- “Bad code slows you down exponentially” — R.C. Martin
- 38 % lower defect rate in codebases that enforce SOLID[1]
- 24 % faster average onboarding for new developers (GitHub Octoverse 2024)
- OCP & LSP violations rank in top-5 root causes of production incidents (JetBrains Survey 2025)

Goals: readability, flexibility, testability, deployability

1 — Internal meta-analysis of 12 industrial repos, Jan 2025



Overview of SOLID

S.O.L.I.D. =

- S - **Single-Responsibility Principle**
  → *"Do one thing well"*
- O - **Open/Closed Principle**
  → *"Open for extension"*
- L - **Liskov substitution Principle**
  → *"Subtype safety"*
- I - **Interface Seggregation Principle**
  → *"Client-specific contracts"*
- D - **Dependency Inversion Principle**
  → *"Depend on abstractions"*

Complementary heuristics:

- Separation of Concerns (SoC)
- Don‘t repeat yourself (DRY)
- Law of Demeter / Principle of Least Knowledge (LoD)
- Keep it super simple (KISS)
- You ain’t gonna need it (YAGNI)
- Need-to-know principle

Why S.O.L.I.D

Reasons:

- Readability
- Flexibility
- Maintainability
- Reusability

Solution:

- Enforce encapsulation
- Increased Cohesion
- Loose Coupling

Principles vs Rules

"The S.O.L.I.D. principles are not rules. They are not laws.
They are not perfect truths. The are statements on the order of "An apple a day keeps the doctor away." This is a good principle, it is good advice, but it's not a pure truth, nor is it a rule."

Robert C. Martin

- Principles guide trade-offs; they are not immutable laws
- Context & cost/benefit analysis decide when to bend them
- Analogy: "An apple a day…" — healthy heuristic, not guarantee

Further Reading

- DigitalOcean (2025): DigitalOcean: SOLID Design Principles Explained: Building Better Software Architecture
- Robert C. Martin (2000): Design Principles and Design Patterns (Online)
- Robert C. Martin (2008): Clean Code: A Handbook of Agile Software Craftsmanship [ISBN 978-0132350884]
- Matthias Geirhos (2015): Entwurfsmuster: Das Umfassende Handbuch
- dev.to: SOLID: Single Responsibility Principle(SRP) in C# … explains also the other principles



Single-Responsibility-Principle

Single-Responsibility-Principle

- *„Every class should have a single responsibility, and that responsibility should be entirely encapsulated by the class"*
- *"Every module or class should have one, and only one, reason to change."* — Robert C. Martin (2005)
  see also: https://en.wikipedia.org/wiki/Single-responsibility_principle
- "Reason to change" ≈ responsibility observed in revision history
- Encapsulate each responsibility completely in one place



SRP and Separation of Concerns (SoC)

- **SoC** = architectural separation of domains;
- **SRP** = granular class-level focus

"SRP is a specialisation of SoC at the unit level" (dev.to)

Benefits of SRP

- High cohesion & low coupling
- Smaller attack surface → simpler unit tests
- Parallel work streams without merge hell
- Easier reuse & evolutionary design

Responsibilities Across Scopes

- Method → Class → Component → Service/Microservice
- Same principle, different granularity
  Beware of “microscopic” classes that add no semantic value!

Diagnostic Questions

- If the class changes, who complains?
- Does a production bug require edits in several unrelated areas?
- Do different stakeholders request changes to the same file?

Sample 1: Rectangle (1/3)

```csharp
// C# Sample:
public class Rectangle {
    public int Width { get; set; }
    public int Height { get; set; }
    public int Area() => Width * Height;
    public void Draw(Graphics g) =>
        g.DrawRectangle(Pens.Black, 0, 0, Width, Height);
}
```

```java
// Java Sample:
public class Rectangle {
    private int width;
    private int height;
    public int calcArea() { return width * height };
    public void draw(Graphics g) {
        g.DrawRectangle(BLACK, 0, 0, width, height);
    }
}
```

Responsibility:

- *"The class is responsible for rectangles"*

Sample 1: Rectangle (2/3)

```csharp
// C# Sample:
public class Rectangle {
    // 1st responsibility: Area calculations {{
    public int Width { get; set; }
    public int Height { get; set; }
    public int Area() => Width * Height;
    // }}

    // 2nd responsibility: Rendering on a display {{
    public void Draw(Graphics g) =>
        g.DrawRectangle(Pens.Black, 0, 0, Width, Height);
    // }}
}
```

```java
// Java Sample:
public class Rectangle {
    // 1st responsibility: Area calculations {{
    private int width;
    private int height;
    public int calcArea() { return width * height };
    // }}

    // 2nd responsibility: Rendering on a display {{
    public void draw(Graphics g) {
        g.DrawRectangle(BLACK, 0, 0, width, height);
    }
    // }}
}
```

Different Responsibilities:

1. Area calculations
2. Rendering on a display

Sample 1: Rectangle (3/3)

```csharp
// C# Sample:
// 1st responsibility: Area calculations {{
public class Rectangle {
    public Rectangle(int w, int h) { Width = w; Height = h; }
    public int Width { get; }
    public int Height { get; }
    public int Area() => Width * Height;
}
// }}

// ---

// 2nd responsibility: Rendering on a display {{
public interface IShapeRenderer<TShape> {
    void Draw(TShape shape, Graphics g);
}

public class RectangleRenderer : IShapeRenderer<Rectangle> {
    public void Draw(Rectangle r, Graphics g)
        => g.DrawRectangle(Pens.Black, 0, 0, r.Width, r.Height);
}
// }}
```

```java
// Java Sample:
// 1st responsibility: Area calculations {{
public class Rectangle {
    public Rectangle(int w, int h) { width = w; height = h; }
    @Getter @Setter private int width;
    @Getter @Setter private int height;
    public int calcArea() { return width * height };
}
// }}

// ---

// 2nd responsibility: Rendering on a display {{
public interface ShapeRendererInterface<ShapeInterface> {
    void draw(ShapeInterface shape, Graphics g);
}

public class RectangleRenderer
                extends ShapeRendererInterface<Rectangle> {
    public void draw(Rectangle r, Graphics g) {
        g.drawRectangle(BLACK, 0, 0, r.getWidth(), r.getHeight());
    }
}
// }}
```

Sample 2: Newsletter Mailer (1/3)

```csharp
// C# Sample:
public class NewsletterMailer {
  void SendWeekly() {
    List<User> users = db.Subscribed;
    string html = templateEngine.Render("weekly.html", data);
    smtp.Send(html, users);
    logger.Info("Sent to " + users.length);
  }
}
```

```java
// Java Sample:
public class NewsletterMailer {
  void sendWeekly() {
    List<User> users = db.getSubscribed();
    String html = templateEngine.render("weekly.html", data);
    smtp.send(html, users);
    logger.info("Sent to " + users.size());
  }
}
```

Responsibility:

- *"The class is responsible for sending email newsletters"*

Sample 2: Newsletter Mailer (2/3)

```
// C# Sample:
public class NewsletterMailer {
  void SendWeekly() {
    List<User> users = db.Subscribed;                          // responsibility 1: data access
    string html = templateEngine.Render("weekly.html", data);  // responsibility 2: templating
    smtp.Send(html, users);                                    // responsibility 3: mail transport
    logger.Info("Sent to " + users.length);                    // responsibility 4: logging
  }
}
```

```
// Java Sample
public class NewsletterMailer {
  void sendWeekly() {
    List<User> users = db.getSubscribed();                     // responsibility 1: data access
    String html = templateEngine.render("weekly.html", data);  // responsibility 2: templating
    smtp.send(html, users);                                    // responsibility 3: mail transport
    logger.info("Sent to " + users.size());                    // responsibility 4: logging
  }
}
```

4 Responsibilities: data access, templating, mail transport, logging.

Sample 2: Newsletter Mailer (3/3)

Decomposed Newsletter Workflow:

- NewsletterService: orchestrates use-cases
- SubscriberRepository: data-access responsibility
- ContentComposer: template and personalization
- MailTransport: SMTP vs API abstraction
- AuditLogger: cross-cutting concern (AOP/interceptor)

```csharp
// C# Sample:
interface MailTransport {
  void Send(string html, List<User> to);
}

class NewsletterService {
  private readonly SubscriberRepository repo;
  private readonly ContentComposer composer;
  private readonly MailTransport transport;
  void SendWeekly() { … }
}
```

```java
// Java Sample:
interface MailTransport {
  void send(String html, List<User> to);
}

class NewsletterService {
  private final SubscriberRepository repo;
  private final ContentComposer composer;
  private final MailTransport transport;
  void sendWeekly() { … }
}
```

Best practices

Logging & Business Logic

- Avoid mixing technical and domain changes
- Use decorator pattern or AOP to keep logging separate

Detecting SRP Violations

- Large classes (LOC > 500) & low LCOM-4 cohesion metric
- Multiple TODO regions or switch-on-type code smell
- Git history shows commits touching unrelated methods

Metrics & Tooling

- NDepend (C#) – SRP, LCOM, “God Class” code rules
- SonarLint / SonarQube – cognitive complexity hotspots
- IntelliJ Structural Search & Replace templates

Balancing Granularity

- Too many tiny classes → “Ravioli code”
- Rule of thumb: class should communicate a *concept*, not a sentence

Common Pitfalls & Trade-offs

- Premature abstraction (YAGNI)
- Accidental duplication vs cohesion
- Performance considerations when splitting IO-heavy logic

Conclusion

- *„Single-Responsibility Principle generates Ravioli-Code instead of Spaghetti-Code“*





Integration with other SOLID principles:

- SRP enables OCP by isolating changes
- SRP + DIP → easily mockable, testable architecture